

Code Review 紀錄內容

更新日期：2024/8/14

日期	CR 人員	題號	備註
2024/08/09	劉昱志	第 1 題	
1. 刪除沒有用到的程式碼	原本的想法是希望 j 可以作為計步器讓裡面的迴圈跑 9 次(出現 9 行)，然後 k 作為乘數。但變數 j 在打印出來的答案中並沒有用到，且觀察外層迴圈中 j 與 k 的關係，可發現 j 與 k 可以結合，以此精簡程式碼。此外，中通常不會用到兩個變數。 修改前： <pre>for (int j = 0, k = 1; j < 9; j++, k++) { for (int i = 2; i < 10; i++) { System.out.printf("%s%s=%s\t", i, k, (i * k)); } System.out.println(); }</pre> 修改後： <pre>for (int k=1 ; k < 10; k++) { for (int i = 2; i < 10; i++) { System.out.printf("%s%s=%2s\t", i, k, (i*k)); } System.out.println(); }</pre>		
2024/08/09	劉昱志	第 1 題	

2. 打印出的答案需對齊
以符合題目要求

題目要求乘積須向右對齊，需改用%2s 讓乘積為個位數的數字也可以向右對齊
修改前：

```
for (int j = 0, k = 1; j < 9; j++, k++) {  
    for (int i = 2; i < 10; i++) {  
        System.out.printf("%s*%s=%s\t", i, k, (i * k));  
    }  
    System.out.println();  
}
```

修改後：

```
for (int k=1 ; k < 10; k++) {  
    for (int i = 2; i < 10; i++) {  
        System.out.printf("%s*%s=%2s\t", i, k, (i*k));  
    }  
    System.out.println();  
}
```

```
2*1= 2  3*1= 3  4*1= 4  5*1= 5  6*1= 6  7*1= 7  8*1= 8  9*1= 9  
2*2= 4  3*2= 6  4*2= 8  5*2=10  6*2=12  7*2=14  8*2=16  9*2=18  
2*3= 6  3*3= 9  4*3=12  5*3=15  6*3=18  7*3=21  8*3=24  9*3=27  
2*4= 8  3*4=12  4*4=16  5*4=20  6*4=24  7*4=28  8*4=32  9*4=36  
2*5=10  3*5=15  4*5=20  5*5=25  6*5=30  7*5=35  8*5=40  9*5=45  
2*6=12  3*6=18  4*6=24  5*6=30  6*6=36  7*6=42  8*6=48  9*6=54  
2*7=14  3*7=21  4*7=28  5*7=35  6*7=42  7*7=49  8*7=56  9*7=63  
2*8=16  3*8=24  4*8=32  5*8=40  6*8=48  7*8=56  8*8=64  9*8=72  
2*9=18  3*9=27  4*9=36  5*9=45  6*9=54  7*9=63  8*9=72  9*9=81
```

2024/08/09

劉昱志

第 2 題

3. 當數值有可能會重複時，不要使用自訂的變數作為初始值以及條件。

迴圈若以變數作為初始值，有可能會遇到隨機數重複的問題，打印出來的結果不保證一定是 6 個。改成使用 set 的長度作為迴圈條件。因為沒辦法確定迴圈要跑幾次（有可能遇到隨機數重複），所以改使用 while 迴圈。

修改前：

```
public static void main(String[] args) {
    Random randNum = new Random();
    Set<Integer> orderSet = new TreeSet<>();
    Set<Integer> unorderedSet = new HashSet<>();
    for (int i = 0; i < 6; i++) {
        int randomLotto = randNum.nextInt(49) + 1;
        unorderedSet.add(randomLotto);
        orderSet.add(randomLotto);
    }
```

修改後：

```
public static void main(String[] args) {
    Random randNum = new Random();
    Set<Integer> orderSet = new TreeSet<>();
    Set<Integer> unorderedSet = new HashSet<>();

    while (unorderedSet.size() < 6) {
        int randomLotto = randNum.nextInt(49) + 1;
        unorderedSet.add(randomLotto);
        orderSet.add(randomLotto);
    }
```

2024/08/09

劉昱志

第 3 題

4. 子類別中使用到父類別的方法用 super

在子類別的方法中，如要使用到父類別的方法時，建議使用 super 的方式，貫徹繼承的概念。

修改前：

```

@Override
public void printInfo() {
    System.out.printf("薪資單\n");
    System.out.printf("姓名: %s 工作部門: %s\n", getName(), getDepartment());
    System.out.printf("月薪: %s\n", getSalary());
    System.out.printf("業績獎金: %s\n", getBouns());
    System.out.printf("總計: %s\n", payment);
}

```

修改後：

父類別

```

@Override
public void printInfo() {
    System.out.printf("薪資單\n姓名: %s 工作部門: %s\n月薪: %s\n", getName(), getDepartment(), getSalary());
}

```

子類別

```

@Override
public void printInfo() {
    super.printInfo();
    System.out.printf("業績獎金: %s\n總計: %s\n", getBouns(), payment);
}

```

2024/08/09

劉昱志

第 3 題

5. 將打印的結果寫在同一行

盡量將要打印的東西寫在同一行中，避免一樣的程式碼重複書寫。

修改前：

```

@Override
public void printInfo() {
    System.out.printf("薪資單\n");
    System.out.printf("姓名: %s 工作部門: %s\n", getName(), getDepartment());
    System.out.printf("月薪: %s\n", getSalary());
    System.out.printf("業績獎金: %s\n", getBouns());
    System.out.printf("總計: %s\n", payment);
}

```

修改後：

```

@Override
public void printInfo() {
    super.printInfo();
    System.out.printf("業績獎金: %s\n總計: %s\n", getBouns(), payment);
}

```

2024/08/09	劉昱志	第 4 題	
6. 刪除沒有作用的程式碼	確認每一行程式碼都是有作用的，沒有用的程式碼不要留存。 修改前：		

```

File f = new File("C:\\Users\\Admin\\Desktop\\HRmain.csv");
FileWriter fw = null;
BufferedWriter bw = null;
FileOutputStream out = null;
OutputStreamWriter osw = null;

try {
    fw = new FileWriter(f);
    bw = new BufferedWriter(fw);
    out = new FileOutputStream(f);
    byte[] uft8bom = { (byte) 0xef, (byte) 0xbb, (byte) 0xbf };
    out.write(uft8bom);
    osw = new OutputStreamWriter(out, StandardCharsets.UTF_8);
    bw = new BufferedWriter(osw);
}

```

修改後：

```

File file = new File("C:\\Users\\Admin\\Desktop\\HRmain.csv");

// UTF-8 BOM 有些文字編輯器需要透過BOM來識別文件是UTF-8編碼
byte[] utf8bom = { (byte) 0xef, (byte) 0xbb, (byte) 0xbf };

// 有中文的檔案轉檔不能只用BufferedWriter,FileWriter
try (FileOutputStream fos = new FileOutputStream(file);
    BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(fos, StandardCharsets.UTF_8))) {

    // 寫入BOM到文件開頭
    fos.write(utf8bom);
}

```

結果都需要打印出 employee.getName(),所以先將其建立成一個變數，需要用到時就不需要重複再打一次。

修改前：

```
for (Employee employee : employeeList) {  
    if (employee instanceof Sales) {  
        Sales sales = (Sales) employee;  
        bw.write(employee.getName() + "," + sales.getPayment());  
        bw.newLine();  
    } else {  
        Supervisor supervisor = (Supervisor) employee;  
        bw.write(employee.getName() + "," + supervisor.getPayment());  
        bw.newLine();  
    }  
}
```

修改後：

```
for (Employee employee : employeeList) {  
    String name = employee.getName();  
    if (employee instanceof Sales) {  
        Sales sales = (Sales) employee;  
        bw.write(name + "," + sales.getPayment());  
        bw.newLine();  
    } else {  
        Supervisor supervisor = (Supervisor) employee;  
        bw.write(name + "," + supervisor.getPayment());  
        bw.newLine();  
    }  
}
```

7. 把要重複使用到的東西設成變數

2023/07/27

劉昱志

第 5 題

8. 建立防呆機制	建立防呆機制，避免使用者輸入非 1-12 之間的數字。 修改前： <pre>System.out.print("請輸入介於1到12間的整數"); // int month = scanner.nextInt(); // 取當前的時間 Calendar calender = Calendar.getInstance(); // Calendar是抽象類別不能new物件，這裡其實是用子類別GregorianCalendar new int year = calender.get(Calendar.YEAR);</pre> 修改後： <pre>// 防呆機制，避免輸入其他數字 if(month < 1 month >12) { System.out.println("輸入錯誤，請輸入介於1到12間的整數"); scanner.close(); return; }</pre>		
2023/07/27	劉昱志	第 5 題	
9. 把要避免的條件寫在 if()內	把要避免的條件 headLine == null 寫在 if 的條件中，並加上 return，若發生此條件便停止程式繼續進行。下方的 while 沒有修改，因為想要 while 迴圈跑到 br.readLine()沒有東西可以讀取為止，亦代表 csv 檔已經讀取完畢。 修改前：		

```
if (headLine != null) {
    header = headLine.split(",");
    String line;
    while ((line = br.readLine()) != null) {
        String[] values = line.split(",");
        if (header.length == values.length) {
            Map<String, String> eachMap = new HashMap<>();
            for (int i = 0; i < header.length; i++) {
                eachMap.put(header[i], values[i]);
            }
            dataList.add(eachMap);
        } else {
            System.out.println("Skipping malformed line:" + line);
        }
    }
}
```

修改後：

	<pre>if (headline == null) { System.out.print("no headerline found."); return; } else { header = headline.split(","); String line; while ((line = br.readLine()) != null) { String[] values = line.split(","); if (header.length == values.length) { Map<String, String> eachMap = new HashMap<>(); for (int i = 0; i < header.length; i++) { eachMap.put(header[i], values[i]); } dataList.add(eachMap); } else { System.out.println("Skipping malformed line:" + line); } } }</pre>		
2024/08/13	劉昱志	第 6 題	
10. Price 欄位排序使用 BigDecimal	以 Price 欄位做排序不建議直接 get("price")做比較，因為是使用泡泡排序法，比較結果可能有誤。先轉換成 BigDecimal 型別後再做比較得到的答案才會是正確的 修改前：		

```
// 資料排序
Collections.sort(dataList, new Comparator<Map<String, String>>() {

    @Override
    public int compare(Map<String, String> o1, Map<String, String> o2) {
        return o2.get("Price").compareTo(o1.get("Price"));
    }

});
```

修改後：

```
// 資料排序
Collections.sort(dataList, new Comparator<Map<String, String>>() {

    @Override
    public int compare(Map<String, String> o1, Map<String, String> o2) {

        BigDecimal price1 =new BigDecimal(o1.get("Price"));
        BigDecimal price2 =new BigDecimal(o2.get("price"));
        return price1.compareTo(price2);
    }

});
```

2024/08/13

劉昱志

第 6 題

11. 不要使用
String.join，改用
StringBuilder

做 String 的連接時，不建議使用 String.join 的方法，建議改使用 StringBuilder
修改前：

```
if (header != null) {
    bw.write(String.join(",", header));
    bw.newLine();
    // 寫入數據
    for (Map<String, String> data : dataList) {
        List<String> inputList = new ArrayList<>();
        inputList.add(data.get("Price"));
        inputList.add(data.get("Type"));
        inputList.add(data.get("Min.Price"));
        inputList.add(data.get("Price"));
        bw.write(String.join(",", inputList));
        bw.newLine();
    }
    System.out.println("成功");
} else {
    System.out.println("There is no headline");
}
```

修改後：

```
// 寫入表頭
if (header == null) {
    System.out.println("There is no headline");
    return;
} else {
    StringBuilder sb = new StringBuilder();

    sb.append(header[0]).append(",").append(header[1]).append(",").append(header[2]).append(",")
      .append(header[3]);
    bw.write(sb.toString()); // 不建議用join，建議改使用StringBuilder
    bw.newLine();
    sb.setLength(0);
    // 寫入數據
    for (Map<String, String> data : dataList) {
        sb.append(data.get("Manufacturer")).append(",").append(data.get("Type")).append(",")
          .append(data.get("Min.Price")).append(",").append(data.get("Price"));
        bw.write(sb.toString());
        bw.newLine();
        sb.setLength(0);
    }
    System.out.println("csv檔輸出成功");
}
```

2024/8/13

劉昱志

7 題

12. 將帳號、密碼、
SQL 語法宣告為常
數化放在上面

將帳號、密碼、SQL 語法宣告為常數化放 main 方法在上面
修改前：

	<pre>public class practice7 { public static void main(String[] args) { String connUrl = "jdbc:oracle:thin:@//localhost:1521/XE"; List<Map<String, String>> carsList= new ArrayList<>(); // 建立連線 try(Connection conn = DriverManager.getConnection(connUrl, "student", "student123456"); PreparedStatement pstmt = conn.prepareStatement("select * from STUDENT.CARS");){</pre> 修改後： <pre>1 select EMP.NAME, EMP.ID from EMP;</pre>		
2024/08/13	劉昱志	第 7 題	
13. 只有這個檔案會使用到的變數改成 private	只有這個檔案會使用到的變數改成 private 修改前： <pre>public static final String conn_url = "jdbc:oracle:thin:@//localhost:1521/XE"; public static final String update_sql = "update STUDENT.CARS set MIN_PRICE = ? and PRICE =? where MANUFACTURER =? and TYPE = ?"; public static final String delete_sql = "delete from STUDENT.CARS where MANUFACTURER =? and TYPE =?"; public static final String username = "student"; public static final String password = "student123456";</pre> 修改後：		

	<pre>private static final String CONN_URL = "jdbc:oracle:thin:@//localhost:1521/XE"; private static final String USERNAME = "student"; private static final String PASSWORD = "student123456"; private static final String INSERT_SQL = "insert into STUDENT.CARS (MANUFACTURER, TYPE, MIN_PRICE, PRICE) values (?, ?, ?, ?)"; private static final String SELECT_SQL = "select * from STUDENT.CARS where MANUFACTURER = ? and TYPE = ?"; private static final String DELETE_SQL = "delete from STUDENT.CARS where MANUFACTURER =? and TYPE =?"; private static final String UPDATE_SQL = "update STUDENT.CARS set MIN_PRICE = ? and PRICE =? where MANUFACTURER =? and TYPE = ?";</pre>		
2024/8/13	劉昱志	第 7 題	
14. 自動生成的字要刪除	Java 自動生成的字要刪掉 修改前： <pre> } catch (SQLException sqle) { // TODO Auto-generated catch block sqle.printStackTrace(); }</pre> 修改後： <pre> } catch (SQLException sqle) { sqle.printStackTrace(); }</pre>		
2023/08/13	劉昱志	第 7 題	
15. 只有這個檔案會使用到的方法改成 private	只有這個檔案會使用到的方法改成 private 修改前：		

	<pre>// 刪除資料方法 public static void doDelete() { PreparedStatement pstmt = null; try (Connection conn = DriverManager.getConnection(CONN_URL, USERNAME, PASSWORD); Scanner scanner = new Scanner(System.in));) {</pre> 修改後： <pre>// 刪除資料方法 private static void doDelete() { PreparedStatement pstmt = null; try (Connection conn = DriverManager.getConnection(CONN_URL, USERNAME, PASSWORD); Scanner scanner = new Scanner(System.in));) {</pre>
2024/08/13	劉昱志 第 7-2 題
16. 不要丟 SQLException 改用 return 的方法	如果用 SQLException 會造成其他人誤以為是 SQL 本身的語法出現問題，建議改成用 return 的方法強制結束程式，若一定要使用的話，建議使用 Exception，讓最下面的 Exceprion 接。 修改前： <pre>if ((pstmt.executeLargeUpdate()) < 1) { throw new SQLException("無此資料"); // 手動丟出，下面的SQLException會接到 //return } conn.commit(); System.out.println("更新成功");</pre> 修改後：

	<pre>if ((pstmt.executeUpdate()) < 1) { System.out.println("更新失敗"); return; }</pre>		
2024/8/13	劉昱志	第 7-2 題	
17. Trycatch 不建議寫兩層	Try catch 寫在一起就好，不需要額外包兩層寫。 修改前：		

```
private static void doInsert() {
    PreparedStatement pstmt = null;
    // try-with-resources自動關閉 connection,scanner
    try (Connection conn = DriverManager.getConnection(CONN_URL, USERNAME, PASSWORD);
        Scanner scanner = new Scanner(System.in));) {

        try {

            conn.setAutoCommit(false);
            System.out.println("請輸入製造商:");
            String manufacturer = scanner.next();
            System.out.println("請輸入類型:");
            String type = scanner.next();
            System.out.println("請輸入底價:");
            String minprice = scanner.next();
            System.out.println("請輸入售價:");
            String price = scanner.next();

            pstmt = conn.prepareStatement(INSERT_SQL);
            pstmt.setString(1, manufacturer);
            pstmt.setString(2, type);
            pstmt.setString(3, minprice);
            pstmt.setString(4, price);
            pstmt.executeUpdate();

            conn.commit();
            System.out.println("新增成功");
        } catch (Exception e) {
            // TODO: handle exception
        }

    } catch (SQLException sqle) {
        System.out.println("新增失敗,原因:" + sqle.getMessage());
        sqle.printStackTrace();
    } finally {
        if (pstmt != null) {
            try {
                pstmt.close();
            } catch (SQLException sqle) {
                sqle.printStackTrace();
            }
        }
    }
}
```

修改後：

```

private static void doInsert() {
    PreparedStatement pstmt = null;
    // try-with-resources自動關閉 connection,scanner
    try (Connection conn = DriverManager.getConnection(CONN_URL, USERNAME, PASSWORD);
        Scanner scanner = new Scanner(System.in)); {

        conn.setAutoCommit(false);
        System.out.println("請輸入製造商:");
        String manufacturer = scanner.next();
        System.out.println("請輸入類型:");
        String type = scanner.next();
        System.out.println("請輸入底價:");
        String minprice = scanner.next();
        System.out.println("請輸入售價:");
        String price = scanner.next();

        pstmt = conn.prepareStatement(INSERT_SQL);
        pstmt.setString(1, manufacturer);
        pstmt.setString(2, type);
        pstmt.setString(3, minprice);
        pstmt.setString(4, price);
        pstmt.executeUpdate();

        conn.commit();
        System.out.println("新增成功");

    } catch (SQLException sqle) {
        System.out.println("新增失敗, 原因:" + sqle.getMessage());
        sqle.printStackTrace();
    } finally {
        if (pstmt != null) {
            try {
                pstmt.close();
            } catch (SQLException sqle) {
                sqle.printStackTrace();
            }
        }
    }
}

```

2024/08/13	劉昱志	第 7-2 題	
18. if ... else 改成 switch ... case	把 if ... else 結構改成 switch ... case 讓程式碼更簡潔也較容易閱讀。 修改前： <pre>try (Scanner scanner = new Scanner(System.in)) { System.out.println("請選擇以下指令輸入:select, insert, update, delete"); String point = scanner.next(); if (point.equals("select")) { doQuery(); } else if (point.equals("insert")) { doInsert(); } else if (point.equals("update")) { doUpdate(); } else if (point.equals("delete")) { doDelete(); } else { System.out.println("輸入錯誤"); } }</pre> 修改後：		

	<pre> System.out.println("請選擇以下指令輸入:select, insert, update, delete"); String order = scanner.next(); // switch case寫 程式好讀 switch (order) { case "select": doQuery(); break; case "insert": doInsert(); break; case "update": doUpdate(); break; case "delete": doDelete(); break; } scanner.close(); </pre>		
2024/08/13	劉昱志	第 4 題	
19. <HW> 自訂排序	有試成功兩個方法 (上傳的程式碼是使用 comparator) 1. 在原本 HRmain 中做排序 <pre> Collections.sort(employeeList, new Comparator<Employee>() { @Override public int compare(Employee o1, Employee o2) { return o2.getDepartment().compareTo(o1.getDepartment()); } } </pre> 2. 在 Employee 中 實作 comparable		

```
@Override
public int compareTo(Employee o) {
    return o.department.compareTo(this.department);
}
```

兩個方法在檔案輸出後都成功排序了!!!

	A	B
1	李中白	65000
2	林小中	80000
3	張志城	35300
4	林大鈞	38200

2024/08/13	劉昱志	第 6-2 題	
20. <HW> 嘗試使用 其他方式將 List<Map>分組	原本使用較複雜的資料結構，改採用 stream 的方式將 List<Map>根據 Map 中相同的 key 做分組 修改前：		

```

// 先建立一個Map,用來裝最後分組的結果 {Acura, [{__=__ , __=__ }, {__=__ , __=__ }]} (這就是carsList)})
// 不要改變原本的資料結構 dataList
Map<String, List<Map<String, String>>> result = new TreeMap<>(); // 有排序要用TreeList

for (Map<String, String> carsMap : dataList) {
    String manufacturer = carsMap.get("Manufacturer");
    result.containsKey(manufacturer); 可以改成這樣判斷
    List<Map<String, String>> carsList = result.get(manufacturer); // 這邊是去判斷manufacturer有無出現過
    if (carsList == null) {
        carsList = new ArrayList<>();
        result.put(manufacturer, (List<Map<String, String>>) carsList);
    }
    carsList.add(carsMap);
}

System.out.printf("%-5s %-5s %-5s %5s\n", "Manufacturer", "TYPE", "Min.PRICE", "Price");

BigDecimal totalminPrice = BigDecimal.ZERO;
BigDecimal totalPrice = BigDecimal.ZERO;
for (Entry<String, List<Map<String, String>>> entry : result.entrySet()) {

    List<Map<String, String>> carList = entry.getValue();

    BigDecimal etotalminPrice = BigDecimal.ZERO; // 進入下一個新迴圈時,又重新開始計算
    BigDecimal etotalPrice = BigDecimal.ZERO;

    for (Map<String, String> carMap : carList) { // entry不同時就會跳出迴圈
        BigDecimal minPrice = new BigDecimal(carMap.get("Min.Price"));
        BigDecimal Price = new BigDecimal(carMap.get("Price"));

        etotalminPrice = etotalminPrice.add(minPrice);
        etotalPrice = etotalPrice.add(Price);
        totalminPrice = totalminPrice.add(minPrice);
        totalPrice = totalPrice.add(Price);
        System.out.printf("%-10s %-10s %6s %s\n", carMap.get("Manufacturer"), carMap.get("Type"),
            carMap.get("Min.Price"), carMap.get("Price"));
    }
    System.out.printf("小計%25s %3s\n", etotalminPrice, etotalPrice);
}

System.out.printf("合計%25s %3s\n", totalminPrice, totalPrice);

```

修改後：

```
/*
 * collect 將流的結果收集到某種集合中(無法確定流的順序，所以在流的地方做排序就好) Collectors.groupingBy收集器，收集了三個參數
 * 1.分組依據 2.Map型態 3.下游收集器 item -> item.get("Manufacturer") 一種lambda表示式
 * item是流中的每個元素(在這裡是每個 Map<String, String>)
 *
 */

Map<String, List<Map<String, String>>> listMap = dataList.stream() // 將列表轉換成流(一種Java8的引入特性，可以更方便的處理集合中的數據)
    .collect(Collectors.groupingBy(item -> item.get("Manufacturer"), TreeMap::new,
        Collectors.toList()));

System.out.printf("%-5s %-5s %-5s %5s\n", "Manufacturer", "TYPE", "Min.PRICE", "Price");

BigDecimal summ = BigDecimal.ZERO;
BigDecimal sum = BigDecimal.ZERO;
for (String aa : listMap.keySet()) {
    BigDecimal totalm = BigDecimal.ZERO;
    BigDecimal total = BigDecimal.ZERO;
    for (Map<String, String> bb : listMap.get(aa)) {

        System.out.printf("%-10s %-10s %6s %s\n", bb.get("Manufacturer"), bb.get("Type"),
            bb.get("Min.Price"), bb.get("Price"));

        BigDecimal minprice = new BigDecimal(bb.get("Min.Price"));
        BigDecimal price = new BigDecimal(bb.get("Price"));

        totalm = totalm.add(minprice);
        total = total.add(price);
    }
    System.out.printf("小計%25s %3s\n", totalm, total);
    summ= summ.add(totalm);
    sum = sum.add(total);
}
System.out.printf("合計%25s %3s\n", summ, sum);
}
```

